⚙️ RATCHET IMPROVEMENT REPORT · March 16, 2026

Before & After

Autonomous code improvement run on @ratchet-run/cli

BEFORE		AFTER
83	→ +3 points	86

Issues: **699** → **658** (41 fixed)

7	4	3	40m 12s
Clicks run	Landed clean	Rolled back	Total time

📋 Ratchet ran 7 clicks in 40m 12s, fixing 41 issues. Performance improved the most (+3 points). 3 clicks were rolled back due to test failures.

Issues Fixed

All issue types detected — counts before and after the run

🏆 **Top 1 Win**

↓ **1** any types

ISSUE	SEVERITY	BEFORE	AFTER	CHANGE
any types	medium	4	**3**	**−1**
functions >50 lines	medium	29	**29**	—
await-in-loop patterns	medium	4	**4**	—
missing auth/security controls	medium	1	**1**	—
dead code indicators (TODO, commented code)	low	8	**8**	—
repeated code lines	medium	351	**354**	**+3**
lines >120 chars	low	256	**259**	**+3**

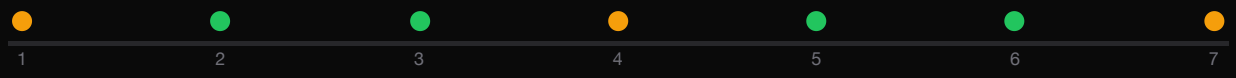
Score Breakdown

Category-by-category improvement

CATEGORY	BEFORE	AFTER	MAX	CHANGE
🧪 Testing	23	23	25	—
🔒 Security	13	13	15	—
📝 Type Safety	15	15	15	—
⚠️ Error Handling	16	16	20	—
⚡ Performance	7	10	10	+3
📖 Code Quality	9	9	15	—

What Ratchet Did

Every change made — 7 clicks, each tested before committing



↩️ Click 1 Rolled back

Fixed issues across 0 files

✅ Click 2 Landed

Fixed issues across 5 files

lib/cli.ts` — added `export function scoreColor(score, max): ChalkInstance` with proper type; imported `ChalkInstance` from chalk, commands/scan.ts` — removed local `scoreColor` function, imported from `cli.ts`, replaced inline ternary with the shared function, core/report.ts` — added `projectName?: string` to `ReportOptions` interface, core/pdf-report.ts` — replaced `(options as any).projectName` with `options.projectName` (type-safe now) +1 more

✅ Click 3 Landed

Fixed issues across 0 files

↩️ Click 4 Rolled back

Fixed issues across 0 files

✅ Click 5 Landed

Fixed issues across 0 files

✅ Click 6 Landed

Fixed issues across 5 files

commands/scan.ts, commands/simulate.ts, commands/tighten.ts, commands/torque.ts +1 more

↩️ Click 7 Rolled back

Fixed issues across 3 files

core/engine.ts, core/history.ts, core/swarm.ts

User Impact Assessment

Persona simulation — how real users would experience these changes

OVERALL SENTIMENT

Users across all skill levels completed the flow with positive or neutral sentiment, but trust and clarity gaps — particularly around rollbacks, output context, and CLI discoverability — created friction at key moments.

Top Pain Points

- No visual confirmation of which project/directory is being scanned or what config was detected (Marcus, Sofia)
- projectName and other ReportOptions not exposed as CLI flags, requiring a config file for basic customization (Priya, Sofia)
- Rollback events show no reason or reassurance, feeling like silent failures (Sofia, implied by Priya's regression concern)
- Score coloring and output terminology (clicks, score thresholds) lack in-context explanation or legend (Marcus, Sofia)
- Scan and torque loop output phases are visually indistinct, making the output hard to parse (Marcus, Sofia)

Top Suggestions

- Print a one-line project context header at the start of every command: project name, detected framework, test command (Marcus, Sofia)
- Expose --project-name (and other ReportOptions fields) as explicit CLI flags on scan and tighten (Priya, Sofia)
- After rollbacks, show a reason line and a first-time reassurance note that rollbacks are expected behavior (Sofia, aligns with Priya's regression concern)
- Add a color legend inline on first score display and document --no-color / NO_COLOR=1 in help output (Marcus, Priya)
- Visually separate scan phase from torque loop with a clear section divider or header (Marcus, Sofia)

Persona Results

Priya Kapoor	(power-user)	neutral
Marcus Chen	(casual)	positive
Sofia Rodriguez	(new-user)	positive